

SEMINAR SUMMARY & STUDY COMPANION

Securing MERN applications

A detailed summary and study companion for B.Tech students.

This guide revisits the Common social-app seminar: how requests travel through a MERN application, where trust boundaries sit, and how to enforce security rules in the server, database and browser. It includes the runnable reference code, review exercises, and free and paid course recommendations.

The central lesson: verify identity, enforce permission, validate input and control the response. React can guide the user; the server must enforce the rule.

What we covered

- Foundations: security goals, request flow, sessions, cookies and JWTs.
- Eight core lessons: ownership, editable fields, XSS, CSRF, JWT verification, login abuse, NoSQL input and public-response privacy.
- Optional extensions: trusted pricing, race conditions and safe link previews.
- Application review: a protected profile route, deployment assumptions and evidence for a working repair.

How to use this guide

Read the seminar explanation first, then clone Common and inspect the code map. Attempt the exercises before reading the answers. Finally, use the project checklist on one route in your own college project.

Inside

- Pages 2-6: foundations and the detailed security lessons.
- Page 7: clone and run the Common reference application.
- Pages 8-10: exercises, worked answers and a project review checklist.
- Pages 11-13: course options, clearly labelled by provider and cost.
- Pages 14-15: a learning plan and primary references.

Common source code and setup instructions on GitHub

<https://github.com/Tejas021/common-mern-security>

Prepared from the seminar materials. Course details were checked on 3 October 2026; access terms and prices may change.

SECTION 02

Foundations and request flow

Security goals: confidentiality, integrity and availability

Confidentiality means private information reaches only permitted readers. Integrity means protected data changes only through permitted operations. Availability means legitimate users can use the service when they need it. A private draft, an account role and an overloaded login endpoint illustrate these three goals in Common.

Follow one request through the MERN stack

React UI -> HTTP request -> Express middleware -> route policy -> MongoDB -> selected response fields -> React rendering

The browser supplies input; it does not establish permission. Express should establish a trusted identity, check the operation against the resource, parse the expected input and construct the database operation. Before returning a response, the server decides which fields may be disclosed. React then displays those values safely.

Authentication and authorization are separate decisions

Authentication answers who is making the request. Authorization answers whether that identity may perform this operation on this resource. A valid login does not imply access to another member's private draft or to an administrator report. Permission needs to be enforced at the server for each relevant action.

Sessions, cookies, browser storage and JWTs

- A server session stores authentication state on the server; a session ID identifies that state.
- A cookie is a browser mechanism for storing and sending a value. That value may be a session ID or a JWT.
- localStorage and sessionStorage are browser storage APIs. JavaScript can read their contents.
- A signed JWT contains claims with a verifiable signature. Signing does not make its payload secret.
- HttpOnly restricts JavaScript access to a cookie. Secure controls HTTPS-only transmission; SameSite affects when a browser attaches it.

The authentication model in Common

Login verifies a password hash and issues a 15-minute JWT in an HttpOnly cookie. The repaired middleware verifies the token and loads the current account. A CSRF value is held in React memory and sent with authenticated writes. Logout revokes the current token ID. There is no refresh-token flow: sign in again when the token expires.

Revocation and account checks use MongoDB. This is a teaching example of a JWT-based login that still depends on server-held state. The repository README describes the local cookie and database assumptions.

SECTION 03

Identity, ownership and permissions

Authentication: establish a trusted identity

In Common, login issued a short-lived JWT in an HttpOnly cookie. The repaired middleware verified the signature, permitted algorithm, issuer, audience and expiry, then loaded the current account. Logout revoked the current token; there was no refresh-token flow.

Decoding a JWT only reads its contents. Verification must happen before trusting its claims. A verified identity still needs permission checks. Real projects also need a suitable password hash, account lifecycle decisions and a way to invalidate access when required.

D01 / Ownership: enforce the resource relationship

Seminar example: a profile update must preserve the boundary between the signed-in member and another member's record.

For self-editing, the server must confirm that the authenticated account owns the target profile. A syntactically valid MongoDB ID proves nothing about permission. A hidden React button is a UI choice; Express must enforce the rule for every relevant request.

Review evidence: an owner can update permitted fields; another ordinary member receives a denial and the stored record stays unchanged. Apply the same reasoning to posts, orders, files and administrative actions.

D02 / Editable fields: define an allowlist

Seminar example: changing a display name must not become a way to change an account's role.

A profile editor should accept only fields it is designed to edit, such as displayName. Role, account ownership, balances and verification state need separate server-controlled workflows. Do not spread arbitrary request bodies into a database update.

Review evidence: documented fields work; unsupported fields cannot alter protected account state. Decide explicitly whether the endpoint rejects unknown fields or ignores them, and test that policy.

D05 / JWT: verify first, authorize next

Seminar example: readable token claims are not trusted identity until verification succeeds.

Use a maintained JWT library with explicit verification settings. Define expiry, logout, key management and account-change behaviour. Keep signing keys on the server. Avoid logging cookies, tokens or authorization headers.

Useful questions for any protected route

- Where does the trusted identity come from?
- What resource does this identity own or have permission to use?
- Which operations and fields are permitted?
- Can permission or account status change during the token lifetime?

SECTION 04

Input and browser security

D07 / NoSQL: validate types before building queries

Seminar example: the moderation illustration showed why an unexpected input shape can change the meaning of a database operation.

Express input is untrusted JSON. A field designed as a search string must be a bounded string, not an arbitrary object. Build database filters from validated values and server-defined structure. Do not let client JSON become MongoDB query operators or update instructions.

Review evidence: valid input works; unsupported types and shapes fail before the database operation; stored data stays unchanged when validation fails. For SQL systems, use parameterized queries rather than constructing SQL from input.

D03 / XSS: keep content as data

Seminar example: comments are user content. Rendering them as active markup changes the browser's trust boundary.

Render ordinary comments through React text interpolation. Avoid raw HTML rendering for user content. If rich text is a deliberate requirement, use a maintained sanitizer configured for permitted elements, attributes and URL schemes.

A content security policy adds another layer. It does not replace safe rendering. HttpOnly reduces script access to authentication cookies but does not prevent every action that unwanted script could perform as a signed-in user.

D04 / CSRF: protect cookie-authenticated writes

Seminar example: the browser can carry authentication cookies with a request initiated from another origin. The write still needs the correct protections.

Browsers attach cookies according to cookie rules. A state-changing route must verify appropriate request origin and CSRF protection for its authentication design. SameSite helps, but evaluate the real deployment topology and do not treat it as the entire defence.

In the seminar, different localhost ports were different origins on the same site. This distinction explains why SameSite=Lax alone was not sufficient in that particular demonstration. Common protected cookie writes with Origin and CSRF checks.

CORS, CSRF and XSS solve different problems

- CORS controls which browser origins may read a response. It is not an authorization system.
- CSRF protection prevents unwanted authenticated changes caused by another origin.
- Safe rendering and context-appropriate encoding address XSS. Backend validation alone does not make arbitrary HTML safe.

Review evidence: comments display as text; an allowed authenticated write works; requests that fail the CSRF policy cannot change state.

SECTION 05

Abuse, privacy and deployment

D06 / Repeated logins: limit abuse thoughtfully

Seminar example: repeated failed sign-ins should meet a deliberate limit instead of consuming unrestricted authentication work.

Protect login and other costly or sensitive actions with suitable rate limits and monitoring. Decide which identity signals to use and how legitimate users recover. Avoid account-enumerating errors. Production systems with multiple server instances usually need coordinated limiter state.

Review evidence: normal sign-in still works; the configured threshold has the intended effect; monitoring records useful events without passwords or tokens. Proxy configuration affects IP-based limits and must match the infrastructure.

D08 / Public API responses: return only public fields

Seminar example: a public profile response must stay public even when the UI shows only a small part of it.

Define the response shape deliberately. A public profile might contain an ID, display name and avatar, rather than every account property. Fields in an HTTP response can be inspected even when React does not display them. Password hashes, reset tokens and private account details do not belong in public responses.

Review evidence: response assertions check the exact permitted fields, including nested objects and error responses.

Before a college project goes live

- Use HTTPS and production-appropriate cookie attributes: HttpOnly, Secure, an appropriate SameSite policy, and deliberate lifetime and scope.
- Keep secrets server-side. Frontend bundles and frontend environment variables are visible to users.
- Restrict database network access, use authentication and least-privilege credentials, and test backup restoration.
- Keep runtime and dependencies supported; review dependency findings and changes before deployment.
- Set appropriate security headers, request size limits, safe error responses and sensitive-action logs.
- Review upload, password-recovery and third-party integration behaviour if the project has those features.

What the local seminar setup teaches

The demonstration used local HTTP, local database settings and a process-local limiter. Its repaired mode demonstrated specific controls; it was not a production readiness guarantee. Use the principles when designing your own deployment.

SECTION 06

Optional extensions and common mistakes

D09 / Pricing: compute trusted values on the server

The server should resolve the product, current price, quantity, discount policy and user entitlement from trusted data. Client-submitted totals are display information. Persist and charge the server-calculated amount.

D10 / Race conditions: preserve invariants atomically

If a reward has one item left, at most one successful claim should consume it. Use an atomic conditional update or an appropriate transaction, rather than a separate read followed by an unchecked write. Account for retries with idempotency where needed.

Review evidence: a local integration test can check the invariant under parallel legitimate claims, and ensure inventory never becomes negative. Repeat requests must not double-charge or double-issue rewards.

D11 / Link previews: constrain outbound requests

A server fetching a user-supplied URL needs a deliberate destination policy, restricted protocols and network egress controls. Address resolution and redirects must be considered as part of that policy. Set timeouts, response-size limits and avoid attaching server credentials to external fetches.

Common illustrated this topic using local fixtures, without outbound network requests. The production design needs controls at both application and network boundaries.

Mistakes to avoid

- Treating a React route guard as server-side authorization.
- Assuming a signed token means every resource is accessible.
- Copying all request fields into a database write.
- Assuming an HttpOnly cookie solves XSS or CSRF by itself.
- Calling CORS a replacement for authentication.
- Believing a clean dependency scan proves the application is secure.
- Fixing a route by rejecting every request; valid user behaviour must still succeed.

A good submission explains the control

For each change, explain the protected asset, the server rule, where the rule is enforced, and the evidence that permitted behaviour works while forbidden behaviour cannot change state.

SECTION 07

Clone and run Common

Common is published as a local student reference with the demonstrated repairs enabled. It includes React, Express, Node.js and MongoDB source, fictional sample accounts, profiles, posts, comments, administrator access and the shop/reward examples.

GitHub repository - clone this codebase

<https://github.com/Tejas021/common-mern-security>

Requirements and first terminal

Install a supported Node.js release satisfying version 22.12 or newer, plus npm. The first package installation and MongoDB binary download require internet. Open a terminal and run:

```
git clone https://github.com/Tejas021/common-mern-security.git
cd common-mern-security
npm ci
npm run build
npm run mongo
```

Second terminal

Keep the MongoDB terminal running. In a second terminal, open the same repository folder and run:

```
npm run seed -- --confirm-reset
npm start
```

Open <http://127.0.0.1:4200> in your browser. Use 127.0.0.1 as the hostname to match the application's origin policy.

Fictional accounts

- hermione@common.demo - member
- ron@common.demo - member
- admin@common.demo - administrator, displayed as Luna
- Password for all three: CommonDemo!2026

Where to study the controls

Read SECURITY-MAP.txt. Authentication and CSRF are in app/server/auth.mjs; permissions are in policies.mjs and routes.mjs; input contracts are in validation.mjs; safe comment rendering is in CommentBody.jsx. app.mjs contains the origin policy, request limits and login limiter.

Reset, stop and troubleshoot

Seeding replaces the local fictional data and invalidates old logins. Stop both terminals with Ctrl+C. Use npm test for the included smoke check. If ports 27017 or 4200 are occupied, stop the other local process. Existing MODE or MONGO_URI environment overrides may need to be removed. The GitHub README includes an optional Docker database setup and registry details.

This is local teaching software. The demonstrated repairs are focused examples; the README explains the deployment, transaction and retry limitations.

SECTION 08

Take-home exercises

Use your own project and fictional test data. These exercises ask you to design and check defensive behaviour; no replay tools are required.

1 / The profile update

A route lets members edit their own display name. Explain: how the server identifies the caller; how it decides whether the caller may edit the profile; which fields it accepts; what changes if login uses a cookie; and which response fields it returns. Write one valid-case and one denial-case test expectation.

2 / The public directory

Design the public response for a People page. Name three allowed fields and three fields that must remain private. Where should the response allowlist live? Explain why hiding fields in React is insufficient.

3 / Comments and search

Your app has plain-text comments and a name search. Specify rendering rules for comments and type/length rules for search. Explain the difference between safe output handling and validation. Describe the expected behaviour for unsupported input types.

4 / One remaining reward

A giveaway has a stock of one. Two eligible members submit legitimate claims at nearly the same time. State the invariant, describe a database operation that preserves it, and define what each member should observe.

5 / Deployment review

List five things you would change when moving from localhost to a deployed project. Include cookies, secrets, database access, rate-limit state and error/logging behaviour. Explain how each change protects a specific asset.

Optional capstone / Review one route

Choose a profile, post or order route. Submit a small change or design note covering identity, authorization, input shape, response privacy, browser protections and deployment assumptions. Include evidence for both accepted and denied behaviour.

Reflection

- What should the client be allowed to choose?
- Which decisions belong exclusively to the server?
- What could change between reading a record and writing it?
- What evidence would convince another developer that the rule works?

SECTION 09

Worked answers

1 / Profile update

Authenticate with verified credentials and load the account where needed. Enforce self-edit ownership on the server. Accept a bounded displayName string through a strict schema; protect privileged fields. For cookie authentication, apply the relevant Origin and CSRF checks. Return a minimal success response or only the documented profile fields. Test an owner success and a non-owner denial with no stored change.

2 / Public directory

An example public contract is id, displayName and avatarUrl. Exclude password hashes, password-reset tokens and private email addresses. Build this contract at the server response boundary. Browser network tools expose response data regardless of what the page renders.

3 / Comments and search

Display ordinary comments as React text. Use a deliberate sanitizer if rich HTML is required. Parse search as a bounded string and construct query structure on the server. Validation ensures the expected input contract; safe rendering prevents content from being interpreted as active markup. Unsupported types should produce a controlled validation error before database work.

4 / One remaining reward

Invariant: stock stays at or above zero, and one available item yields at most one successful claim. A conditional atomic decrement can enforce availability; coordinate associated records with a transaction if needed. One member receives success and the other a clear unavailable result. Idempotency should handle repeat submissions without duplicate rewards.

5 / Deployment review

Enable HTTPS and secure cookie attributes; place private secrets in server-side secret configuration; authenticate and restrict database access; coordinate rate limits across application instances; and return safe errors while recording useful events without credentials. Document trusted proxy and origin assumptions.

Capstone assessment: 10 points

- 2 points: authenticated identity and server-side resource permission are explained.
- 2 points: input shape and protected fields are explicit.
- 2 points: response privacy and relevant browser controls are addressed.
- 2 points: evidence covers success, denial and unchanged data after failure.
- 2 points: deployment assumptions and remaining limitations are stated.

A correct answer names the security rule and evidence, not just a package. There are several valid implementations when they preserve the same policy.

SECTION 10

Project review checklist

Review one feature at a time. Record PASS, NEEDS WORK or NOT APPLICABLE, plus an evidence link or test name. A tick without evidence is not a review.

- Identity: credentials are verified; expiry and logout behaviour match the design.
- Permissions: every protected read/write checks the caller and the target resource on the server.
- Fields: request parsing accepts only documented fields and preserves server-controlled state.
- Queries: database filters and updates are assembled from validated values and trusted structure.
- Rendering: user comments render safely; rich content has a deliberate sanitization policy.
- Cookie writes: Origin, SameSite and CSRF choices match the application topology.
- Abuse: sensitive endpoints have suitable limits and useful monitoring.
- Responses: public data is allowlisted; errors do not expose private implementation details.
- Business rules: prices and privileges come from trusted server-side data.
- Concurrency: operations preserve stock, balance and uniqueness invariants under retries.
- Outbound requests: link previews and integrations enforce a destination and egress policy.
- Production: HTTPS, cookie settings, secrets, database access and runtime support are reviewed.
- Verification: legitimate behaviour still succeeds; failed checks cause no protected state change.

Evidence template

Feature / asset: _____

Security rule: _____

Enforcement location: _____

Accepted-case result: _____

Denied-case result and stored state: _____

Deployment assumptions / remaining work: _____

Definition of done for this exercise

A classmate can read your evidence and explain why the route accepts the intended request and denies a forbidden one. This is a focused learning milestone, not a comprehensive security certification.

SECTION 11

Recommended free secure-coding courses

Our selection for this seminar: beginner accessibility, application-security relevance, MERN transfer, practice, maintenance and cost. These are editorial recommendations based on published curricula, not claims that we completed each course.

1 / Hacksplaining

Provider: Hacksplaining website / Free for personal use

Best quick introduction for this seminar cohort. Free for personal use; the provider lists 45+ interactive secure-coding lessons, usually 15-30 minutes each, with quizzes and examples in several languages including JavaScript. Select the topics connected to the seminar instead of trying to finish everything. Team dashboards and reporting are separate paid features.

Official lessons and personal-use terms

<https://www.hacksplaining.com/>

2 / Snyk Learn: JavaScript security + OWASP API Security Top 10

Provider: Snyk Learn / Free

Best immediate fit for students already building MERN projects. Free secure-development lessons; select the JavaScript security catalogue, not product-administration training. The catalogue lists OWASP Top 10 at about 2h 30m and API Security Top 10 at about 2h 40m. Budget more time to apply the controls in your code. Snyk lists quizzes and certificates in its free education plan.

JavaScript security catalogue

<https://learn.snyk.io/catalog/security-education/javascript/>

Free plan and certificate information

<https://snyk.io/platform/snyk-learn/>

3 / OpenSSF Developing Secure Software (LFD121)

Provider: Linux Foundation Training, developed by OpenSSF / Free

Best structured free secure-development course in this comparison. Covers requirements, secure design, dependency reuse, input/output handling, verification and threat modelling. Some software-development knowledge is expected. The Linux Foundation lists 16-20 hours and 90 days of course access; OpenSSF estimates 14-18 hours. Budget about 20 hours plus project practice. OpenSSF confirms the course and completion certificate are free; passing the final exam is required, and the certificate is valid for two years.

Enrollment, syllabus and current access terms

<https://training.linuxfoundation.org/training/developing-secure-software-lfd121/>

OpenSSF certificate conditions

<https://openssf.org/training/courses/>

How to choose

Quick start: Hacksplaining -> Snyk -> apply the controls to your own project. Add LFD121 as the structured follow-up. PortSwigger and additional foundation options are on the following page. Snyk lessons vary in language and ecosystem coverage; check each lesson rather than assuming every example uses Express and MongoDB.

SECTION 12

More free learning and practice

PortSwigger Web Security Academy

Provider: PortSwigger / Free training

A strong supplement for understanding web-security problems and the controls that prevent them. Start with the introductory learning guidance and the topics from the seminar. The Academy is stack-independent and includes interactive labs; the professional certification is separate from free learning.

Academy introduction

<https://portswigger.net/web-security/getting-started>

Full Stack Open

Provider: University of Helsinki / Free

Useful if React, Express, MongoDB, backend structure or testing need strengthening. The course and completion certificate are free subject to the required exercises. It expects programming, database and Git foundations; this is a substantial development course rather than a dedicated security qualification.

Course, prerequisites and certificate conditions

https://fullstackopen.com/en/part0/general_info/

Introduction to Cybersecurity

Provider: Cisco Networking Academy / Free

A beginner foundation in cybersecurity and privacy, approximately six hours, with a digital badge listed by the provider. Choose it if you need the broader vocabulary before specialising in application security.

Cisco course

<https://www.netacad.com/courses/introduction-to-cybersecurity>

CS50 Introduction to Cybersecurity

Provider: Harvard CS50 / Free learning; paid edX verification is optional

Five weeks of lecture-based material plus a final project. A free CS50 certificate is available after meeting the requirements. An edX verified certificate is a separate paid choice.

Harvard course and certificate FAQ

<https://cs50.harvard.edu/cybersecurity/faqs/>

Optional practice references

OWASP NodeGoat is a Node-focused tutorial and training application. OWASP Juice Shop provides challenge-based web-security learning and uses Angular. These are additional practice formats, not React implementation courses.

OWASP NodeGoat

<https://github.com/OWASP/NodeGoat>

OWASP Juice Shop

<https://owasp.org/projects/juice-shop>

SECTION 13

Optional paid courses

Paid courses are optional. Compare the learning goal, preview lessons, example versions and actual checkout price. Public ratings are snapshots of learner sentiment, not a technical security review.

OWASP Top 10 2025: Web App Security for Beginners (No Code)

Platform: Udemy | **Instructor:** Soerin Bipat | **Paid**

A vocabulary and conceptual refresher, approximately 1h 38m. The checked listing showed 4.5/5 from about 50,000 ratings and a July 2026 update. It covers concepts rather than MERN implementation, and retains material from earlier OWASP versions alongside updates.

Udemy course

<https://www.udemy.com/course/web-application-security-for-absolute-beginners-no-coding/>

Application Security - The Complete Guide

Platform: Udemy | **Instructor:** Derek Fisher | **Paid**

Approximately 12h 33m covering OWASP, threat modelling and security through the software development lifecycle. The checked listing showed 4.4/5 from 16,679 ratings and a July 2026 update. Good for structured application-security design; examples are broader than the MERN stack.

Udemy course

<https://www.udemy.com/course/application-security-the-complete-guide/>

Node.js, Express, MongoDB & More: The Complete Bootcamp

Platform: Udemy | **Instructor:** Jonas Schmedtmann | **Paid**

Approximately 42h 12m of backend project work, with authentication and security sections. The listing showed 4.7/5 from 30,406 ratings and a November 2024 update. It includes Pug-based server rendering and older runtime examples. Use current official documentation when adapting versions or security recipes.

Udemy course

<https://www.udemy.com/course/nodejs-express-mongodb-bootcamp/>

Budget and other options

Indian checkout prices were not reliably visible during research. A suggested personal budget is up to INR 800 for one optional course during a sale; this is not a verified price. The recommended free path costs INR 0.

Ronan Takizawa's Web Security for Web Developers is about 30 minutes, so it is a brief overview. Brad Traversy's API Masterclass lists an October 2019 update and uses some dated security packages. Neither is our first purchase recommendation for this cohort. The supplied TryHackMe OWASP room currently requires Premium or Max.

SECTION 14

A four-week project learning plan

Suggested effort: five to seven hours per week. The four-week project plan below is our learning suggestion, not a provider completion estimate. Extend it to six weeks if you also complete LFD121.

Week 1 - Understand the feature and its trust boundaries

Study selected Hacksplaining lessons related to the seminar. Draw your own project's request flow. Identify private assets, the authenticated caller and the rules for one profile, post or order route. Use Cisco or CS50 if broader security foundations are still unclear.

Deliverable: a one-page identity, permission and data-privacy policy for one feature.

Week 2 - Apply strict contracts and permissions

Work through the Snyk JavaScript and API-security lessons that fit your feature. Add or review server-side ownership checks, permitted fields and response allowlists. Use explicit input shapes to keep client data separate from database instructions.

Deliverable: a small code change with evidence for accepted and denied behaviour.

Week 3 - Review browser-facing behaviour

Use introductory PortSwigger material and the prevention references. Review comment rendering, cookie configuration, origin assumptions and CSRF protection. Explain which control addresses which problem; CORS, CSRF and safe rendering serve different purposes.

Deliverable: a browser-security design note plus targeted verification.

Week 4 - Complete the project review

Use the checklist, review dependency findings and record deployment assumptions. If your feature involves stock or pricing, define the business rule and concurrency invariant. Ask a classmate to review whether your evidence establishes the rule.

Deliverable: a short report or README showing controls, evidence and remaining work.

Continue with structured learning

Complete OpenSSF LFD121 as a follow-up, budgeting about 20 hours plus practice. Choose one paid course only if its format helps you. The goal is to explain and implement security decisions, not to collect every available certificate.

Evidence of learning

- Name the protected asset and the permission rule.
- Show where the server enforces the rule.
- Explain the accepted input and returned fields.
- Demonstrate intended behaviour and a meaningful denial.
- Document deployment assumptions and limitations.

SECTION 15

Primary references and sources

These free references complement the courses. Use the current documentation when implementing controls; a library name alone is not a security policy.

Express: production security best practices

<https://expressjs.com/en/advanced/best-practice-security/>

OWASP: authentication

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP: password storage

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP: session management

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

OWASP: authorization

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

OWASP: NoSQL security

https://cheatsheetseries.owasp.org/cheatsheets/NoSQL_Security_Cheat_Sheet.html

OWASP: XSS prevention

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

OWASP: CSRF prevention

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP: SSRF prevention

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

MDN: cookie attributes

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie>

Seminar and code sources

The explanation follows the seminar README, demo list, exercise, answer key, student checklist and running order. D01-D08 are the core lessons. D09-D11 are optional extensions; the planned running order does not establish which optional demonstrations were shown live.

Common student reference repository

<https://github.com/Tejas021/common-mern-security>

Course research and access conditions

Provider pages support the platform names, syllabus descriptions, published durations and access conditions. Course ratings, updates and costs were checked on 3 October 2026 and can change. Linux Foundation lists 16-20 hours for LFD121; OpenSSF gives a 14-18-hour estimate and confirms the free completion certificate. Keep provider links as the source of current enrollment terms.

OpenSSF course and certificate information

<https://openssf.org/training/courses/>